Squishing Over It

GIRARDIN Baptiste - MARMOND Colin
JEMEL Elyes - VIVARES Edouard
IGR Course - Telecom Paris - Jun. 26th 2026

ABSTRACT

This paper aims to explain the thought process that allowed us to make a first version of the video game called "**`Squishing over it`**". We will dive into the details of the creative process as well as some technical details to explain how we made our squishy main character.

INTRODUCTION AND PROJECT MAKING

The initial proposal for this project was to do a 2D or 3D video-game relying on rigid-body dynamics. After thinking together we wanted to explore the physical game-play a bit further by also implementing soft-body dynamics. As difficult as soft bodies can be for a game, we had to put forward clever tricks to make it work in realtime. This led us through different phases for the making of this project. We first started thinking about the core game-play to narrow down the mechanics we wanted to add. Then after having the mechanics and technical parts implemented we moved towards level design and finishing a somewhat playable game prototype.

GAME CONCEPT & MOTIVATIONS

Inspirations

We started on a blank board and discussed the "What" question. We gathered our experiences and visions for the game and our ideas converged towards a platformer game inspired by some video games we knew: Jellycar, Human Fall Flat. We really wanted to integrate the funny jelly simulation of Jellycar into our universe. That is why our main character behaves like a jelly squishy creature. Human Fall Flat inspired us for its universe, art style and ragdoll-based game mechanics.

Concept

After keeping only the essential parts of our design, we ended up with this idea:

The game would be a low-poly 3D puzzle platformer where a squishy ball (deformable by the player and its environment) would wander and solve puzzles to reach the top of a mountain. Not only would the level be funny for the player, but the physics would be pleasant to play with as well. We chose to make it low-poly for performance and artistic reasons.

Game design

We brainstormed ideas for obstacles that could be interesting in regards to the squishy body mechanics. We thought that playing with the squishy physics to interact with other objects would be fun. Thus we added rigid-soft bodies interactions.

The rigid-bodies would take multiple forms in the game-play and so we first drafted some concepts on paper to get a rough idea of how we could use them as well as some ideas for designs for the main character. Here's what that looked like:

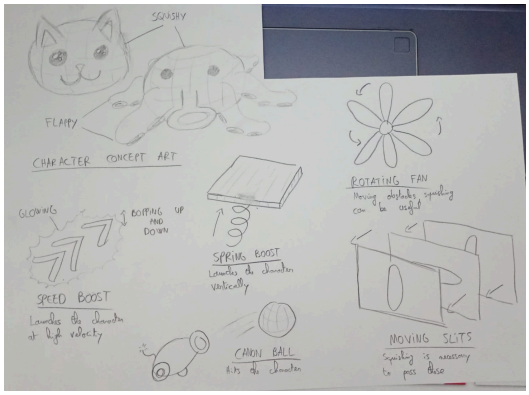

Figure 1: Concept arts for the obstacles and character

The cannons and rotating fans are the most straight forward: they are simply objects that exist and move in the world that the player has to avoid in order not to be ejected and fall into the void. The moving slits however are part of the second category of obstacles: the one that include the squishiness of the character as an inherent feature to beat it. In this case, the player has to squish the ball in the right orientation to go through the horizontal or vertical slit unharmed, otherwise they will be ejected.

Finally we see a third type of objects that aren't really obstacles but more like interactive objects. The speed boost will increase the player's velocity in a certain direction, and the spring will launch the player upward at a certain distance, allowing them to access otherwise impossible to reach areas.

TECHNICAL DETAILS

The game relies deeply on physics simulations. The ball reacts to its surroundings and also impacts the moving rigid bodies around. For both the soft-body and rigid-body solutions, we use a semi-implicit Euler integrator to apply the forces. We chose this integrator for its simplicity and stability compared to the explicit and implicit Euler integrators.

Soft-Body Simulation

Soft-body simulation is known to be quite intricate when targeting real-time and physical accuracy. Therefore, we opted for a custom, visually pleasant but unrealistic solution based on spring dynamics. Moreover, we make use of the spherical geometry of the character by completely ignoring its rotation, thus simplifying the simulation.

Architecture

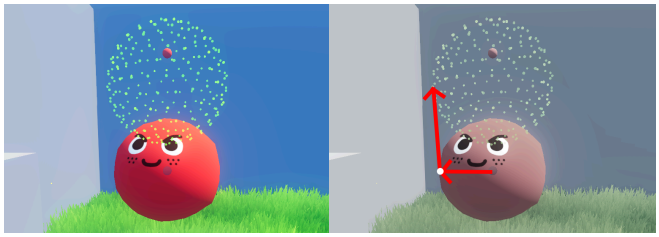

Figure 2: *On the left, the skeleton is the set of green points. On the right, the arrows are the spring forces applied to the vertex at the white circle.*

The main idea is to have two arrays of positions at iteration t : one for the vertices of the mesh $(\boldsymbol{x}_i^t)_i$, and one for its *skeleton* $(\boldsymbol{y}_i^t)_i$ acting as a non-deformable sphere (see Figure 2).

For each vertex, two springs are assigned (Figure 2):

- The *skeleton spring* linking the vertex to its corresponding position in the skeleton, ensuring the rest state to be close to a sphere.
- The *center spring* linking the vertex to the center of the mesh, giving it jelly-like oscillations under deformation.

The formulas of the springs will be detailed in the next section.

With this, the mesh follows the skeleton while trying to conserve its shape. Therefore for now, the skeleton would have an acceleration and velocity completely unrelated to the mesh, leading to divergence when colliding, as explained later. To fix this, most similar implementations rely on linking the center of the skeleton to the center of the mesh, ensuring simple motions [3]. Our approach is different. In order to have more control over the motion, the skeleton is made responsible for the global velocity of the object, while the mesh is made responsible for the local velocities (vertex-wise) of the object. This enables us to have a smooth and stable simulation.
Lastly, to handle collisions and make sure the mesh does not penetrate other meshes, we perform the collision detection and resolution (see Section 3.3) on the mesh directly. We then compute the normal force exerted by the springs on the ground and apply it to the skeleton, allowing it to react to the collision, as shown in the next section.

Forces Applied

Multiple forces are applied to the soft-body in order to achieve visually pleasing results. We will only cover the default forces handled by our engine (see Section 3.4 for the abilities).

- For the global forces, gravity and a friction force are applied to the skeleton to model simple falling motion.

$$\boldsymbol{F}_{\text{glob}}^{t+1} = m.\boldsymbol{g} - \mu.\dot{\boldsymbol{y}}^t$$

- For the local forces, we first compute the two spring forces $\boldsymbol{F}_{\text{ss},i}^{t+1}$ and $\boldsymbol{F}_{\text{sc},i}^{t+1}$ mentioned above, and their related damping forces $\boldsymbol{F}_{\text{ds},i}^{t+1}$ and $\boldsymbol{F}_{\text{dc},i}^{t+1}$

$$\boldsymbol{F}_{\text{ss},i}^{t+1} = -k_{\text{ss}}.(\boldsymbol{x}_i^t - \boldsymbol{y}_i^{t+1}) \qquad \boldsymbol{F}_{\text{ds},i}^{t+1} = -\mu_{\text{ds}}.\boldsymbol{v}_{\text{ss},i}^t$$
$$\boldsymbol{F}_{\text{sc},i}^{t+1} = -k_{\text{sc}}.\frac{\|\boldsymbol{r}_i\|-R}{\|\boldsymbol{r}_i\|}.\boldsymbol{r}_i^t \qquad \boldsymbol{F}_{\text{dc},i}^{t+1} = -\mu_{\text{dc}}.\boldsymbol{v}_{\text{sc},i}^t$$

With $\boldsymbol{v}_{\text{ss},i}^t$ and $\boldsymbol{v}_{\text{sc},i}^t$ respectively the velocities of the skeleton and center springs, $\boldsymbol{r}_i^t = \boldsymbol{x}_i^t - \hat{\boldsymbol{x}}^t$, $\hat{\boldsymbol{x}}^t$ the mean of the vertices, R the original radius of the sphere.

- When colliding at iteration t and vertex i on a surface of normal $\boldsymbol{n}$, here is the global force applied to the skeleton :

$$\boldsymbol{F}_{\text{bounce},i}^t = -\tfrac{1+e}{N}.\min\big(\langle \boldsymbol{F}_{\text{ss},i}^t + \boldsymbol{F}_{\text{sc},i}^t \mid \boldsymbol{n}\rangle, 0\big).\boldsymbol{n}$$

With e the coefficient of restitution of the bounce, N the number of vertices

Custom Tricks

In order to put it all together and to make the results more appealing, we thought of *tricks* customized to our solution.

- The first trick concerns the energy of the body after bouncing. $\boldsymbol{F}_{\text{bounce}}^t$ is computed before the collision pass, therefore the spring forces are inaccurate because it is computed before the mesh is de-penetrated. As a result, the body acquires more energy than it should at every bounce. To fix this, we applied to the skeleton the same translation than the one applied from the de-penetration on the mesh, compensating exactly for the energy miscalculation.

$$\hat{\boldsymbol{y}}^{t+\frac{1}{2}} \leftarrow \hat{\boldsymbol{y}}^{t+\frac{1}{2}} + \left(\hat{\boldsymbol{x}}^{t+\frac{1}{2}} - \hat{\boldsymbol{x}}^t\right)$$

  $\hat{\boldsymbol{x}}^{t+\frac{1}{2}}$ being the value of $\hat{\boldsymbol{x}}$ at iteration t after the collision pass.

- The second trick concerns horizontal collisions. For our gameplay, we want our soft-body to be really soft when bouncing on the ground, but less soft when colliding with the walls. This is because we don't want our body to be able to pass through any hole or cavities, forcing the player to cleverly use abilities shown in Section 3.4. Therefore we adapt the integration and collision steps to solve this constraint by doing this:
  - $\boldsymbol{x}^{t+1} = \dot{\boldsymbol{x}}^{t+1}\Delta t$
    becomes
    $$\boldsymbol{x}^{t+1} = \left(\dot{\boldsymbol{x}}^{t+1} + \alpha \boldsymbol{P}_g \dot{\boldsymbol{y}}^{t+1}\right)\Delta t$$
    $\boldsymbol{P}_g$ being the projection to the ground's plane, and $\alpha \in [0,1]$ a hardness factor.
  - $\boldsymbol{F}_{\text{bounce}}$ becomes $\boldsymbol{F}'_{\text{bounce}} = \beta \boldsymbol{F}_{\text{bounce}}$,
    $\beta \in [1, +\infty[$ being another hardness factor.

  These changes help the horizontal bounces smoothly transition towards rigid collisions.

- Last but not least, we crafted an artificial volume conservation by first computing a vector representing the *mean deformation* of the body:

$$\mathbf{MD} = \frac{1}{N}\sum_i \frac{\max(R - \|\boldsymbol{r^+}_i\|, 0)}{\|\boldsymbol{r^+}_i\|}.\boldsymbol{r^+}_i$$
$$\boldsymbol{r^+}_i = \boldsymbol{r}_i - 2\,\min(\langle \boldsymbol{r}_i \mid \hat{\boldsymbol{n}}\rangle, 0).\hat{\boldsymbol{n}}$$
$$\boldsymbol{r}_i = \boldsymbol{x}_i - \hat{\boldsymbol{x}}$$

$\boldsymbol{r^+}_i$ is $\boldsymbol{r}_i$ flipped to be in the same hemisphere than $\hat{\boldsymbol{n}}$, which is the mean normal vector of the collisions of the $(\boldsymbol{x}_i)_i$. **MD** is in fact the normal vector of the plane of the average compression,

and is scaled by the amount of compression. This vector is used in the formula of $\boldsymbol{F}_{\text{sc},i}^{t+1}$, where R is replaced by

$$R_i = R(1 + \lambda \|\mathbf{MD} \times \boldsymbol{r}_i\|)$$

where $\lambda \in [0, 1]$ is the *squish factor*.

Geometry construction

The topology of the sphere is really important, because it heavily influences the performances with its number of vertices. Moreover, every vertex needs to be evenly spaced in order to have consistent forces across different angles of collisions. Multiple approaches were possible (icosphere, Fibonacci sphere [4]) but most of them had major drawbacks (not enough control on the number of vertices, no obvious triangulation). We thus opted for a geodesic sphere [1]. Its advantages are :

- We have more control the number of vertices than the icosphere, $N \approx 10.l^2$ with l the number of subdivisions by edge.
- The vertices are approximately evenly spaced on the surface.
- The triangulation is similar to the one for the icosphere.

Rigid-Body Simulation

While soft-body dynamics are fun, they are way better when coupled with dynamics objects. Therefore we decided to implement a very simple simulation of cuboid rigid-bodies based on soft-constraints so that the soft-body could interact with them.

Global implementation

The implementation is very similar to most implementations of rigid-bodies. We apply the forces, integrate the linear and angular velocities, and integrate once more for the position and rotation. Often-times, three major challenges appear:

- How to apply forces
- How to compute the inertia tensor
- How to detect collisions (point - triangle, edge - edge)
- How to resolve them

We answer the 2nd and 3rd ones by restraining the obstacles to infinite planes, avoiding edge collision detection, and by only implementing cuboid rigid-bodies, making the intertia tensor very simple to compute.
Furthermore, what is central but well-known in rigid-body simulation is how to apply forces, in particular how to apply impulses. Impulse are responsible for a lot of interactions, such as ground friction or bouncing. The formula is given as such for a force $\boldsymbol{F}$ of direction $\frac{\boldsymbol{F}}{\|\boldsymbol{F}\|} = \boldsymbol{n}$:

$$\boldsymbol{j} = \frac{-\boldsymbol{F}}{m_A^{-1} + \boldsymbol{n} \cdot [(\boldsymbol{I}_A^{-1}(\boldsymbol{r}_A \times \boldsymbol{n}) \times \boldsymbol{r}_A)]}$$

in particular, for the contact forces:

$$\boldsymbol{j} = \frac{-(1+e)(\Delta \boldsymbol{v} \cdot \boldsymbol{n}).\boldsymbol{n}}{m_A^{-1} + m_B^{-1} + \boldsymbol{n} \cdot [(\boldsymbol{I}_A^{-1}(\boldsymbol{r}_A \times \boldsymbol{n}) \times \boldsymbol{r}_A) + (\boldsymbol{I}_B^{-1}(\boldsymbol{r}_B \times \boldsymbol{n}) \times \boldsymbol{r}_B)]}$$

with $\Delta \boldsymbol{v}$ the relative velocity at the contact points, and $\boldsymbol{I}_A$, $\boldsymbol{I}_B$ the inertia tensors.
j is then applied as :

$$\boldsymbol{v}^{t+1} = \boldsymbol{v}^t + \frac{\boldsymbol{j}}{m}$$
$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t + \boldsymbol{I}^{-1}.(\boldsymbol{r} \times \boldsymbol{j})$$

where $\boldsymbol{r}$ is the vector from the center of the rigid-body to the point where the force is applied.

Soft constraints

For de-penetration, we used soft-constraints. The issue with regular de-penetration (simply instantly translating) is the stability issue (constant jittering). Soft-constraints model the rigid-body de-penetration as a mass - quasi-spring system to dampen the resolution [2]. The resulting velocity is then computed as such for the semi-implicit Euler integrator :

$$\boldsymbol{v}_i^{t+1} = \frac{\boldsymbol{v}_i^t - \frac{\beta}{m\gamma}\boldsymbol{x}_i^t}{1 + \frac{\Delta t}{m\gamma}}$$

with β the error feedback and γ the constraint feedback, both acting as softness parameters. As we just said, the rigid-body is modelled as a mass - **quasi-spring** system. With a spring we apply a force to affect the acceleration. Here we are trying to adjust the velocity and the constraint force will be determined to satisfy this velocity constraint. The values of β and γ are chosen to mimic an harmonic oscillator:

$$\gamma = \frac{1}{c + \Delta t k} \qquad \beta = \frac{\Delta t k}{c + \Delta t k}$$

c and k the damping and rigidity factors of the spring.

Soft-body interaction

The last point was handling soft-rigid bodies interactions. The principle is actually very simple:

- The soft-body bounces off the rigid-body at each frame as if it were static
- At the same time, each spring colliding with the rigid-body applies a small impulse at the contact point scaled by its mass and the mass of the rigid-body

This results in a satisfying interaction, but we have to be careful for when the rigid-body and soft-body move in opposite directions. To try to handle that, we offset the position along the normal, with $\boldsymbol{v}_{\text{rb}}$ the velocity of the rigid-body at contact point :

$$\boldsymbol{x}_i \leftarrow \boldsymbol{x}_i + \max(\langle \boldsymbol{v}_{\text{rb}} \mid \boldsymbol{n} \rangle, 0).\frac{\Delta t}{N}.\boldsymbol{n}$$

Collisions

The collision system is fairly simple but still requires some optimization to process in realtime the thousands of potential collisions.

Detection

At each physic step, each vertex has a past position x^t and a new position x^{t+1} that is integrated by the forces applied to the vertex. What we do is trace a 3d segment going from x^t to x^{t+1} and check if it intersects with a triangle. If it does, then we project it back to the surface of the triangle according to the triangle's normal. It is as simple as that, but that comes with a cost, we have to test each vertex with each triangle. If we only have a few triangles that is probably alright, but for a whole scene this is not anymore.

Scene Hierarchy

In order to fix this performance problem, many options are open to us, BVH being the most known algorithm to have. But as our collision system is fairly simple and needs moving objects, we thought that it would be interesting to do a simpler method.

That is why we used a simple list of spheres `(radius, center)`. We only have one squishy sphere that is moving, so at each physic step we discriminate all the colliding objects that are too far from the squishy object. Instead of discriminating objects for each vertex movement, we do it once for the whole squishy object.

To do so, we take the minimal sphere that the set $\left\{x_i^t, x_i^{t+1}, i \in [0, N]\right\}$ can fit in.

So, instead of doing it for each vertex, we do it for the whole squishy ball once. Then we do segment to triangle intersection for each vertex and each triangle given from the pre-intersection phase.

Resolution

Once we have all the collisions detected, we can move the vertex to the optimal position.

This position is calculated by projecting the vertex onto the surface of the colliding triangle and applying a tiny offset (`1e-3`) along the surface normal to prevent it from getting stuck (de-penetration). Once the vertices are repositioned, we resolve the collision physics in three main steps:

- Velocity Dampening: We reduce the vertex's local velocity along the collision normal based on an energy absorption factor to simulate the material's bounciness.
- Global Rebound: We calculate the internal spring forces (pushing outward from the core and skeleton) against the surface to compute a total rebound force, as seen in Section 3.1.2
- Rigidbody Interactions: If we hit a rigid-body, we calculate and transfer an impact force to push it. We also add the moving object's velocity back to our vertices to prevent phasing through mobile platforms.

Finally, we average the data from all colliding vertices to compute a mean collision normal and a ground direction. This global state update ensures the entire squishy object is correctly oriented and deformed for the next simulation step.

Abilities & Interactable objects

The character has different controls. The usual `Forward, Backward, Right, Left, Jump` but also three more controls being `Squish orizontally, Squish vertically, Dash`.

This allows the player to better control the character and go through tight spaces, jump and speed up if needed.

Moreover, the character can interact with its environment. It can jump onto walls and back, play with rigid-body objects. For the fun aspects and to make the game more pleasant, we constrained some rigid-body objects to only rotation or translation or some limited axis. This allows to do all sort of things: a swing, movable cubes to climb on.

GAME LOOP & PLAYABILITY

In order to do an interesting prototype for this game, we had to create a game loop and use HUD (Head Up Display).

Because the game is ought to be a fun game to play while keeping a small timing, we decided to put the time for the level happening, as well as some basic GUI when the player finishes the project.

Technically, in Godot, this was achieved by creating multiple scenes:

- one main scene having all the GUI
- different level scenes

The main scene would load and reload levels as needed so that the user starts with a freshly copied scene. A system of signals allows communication between the scenes.

FURTHER IMPROVEMENTS

While having done a fairly good looking game, our project is far from being deliverable to the end user. The physic system is robust enough, but if we aimed for more compatibility with low-end computers and other game devices we would need to optimize even more the collision detection and the handling of low refresh rates. Moreover, collision detection and resolution on dynamic objects is still not satisfying enough as we still oftentimes pass through the object. The level library is also very small and some user would want to explore the idea more.

CONCLUSION

This project has been a fun and instructive moment for us, we learned or perfected our knowledge on `Godot`'s framework. While having very different backgrounds regarding video game creation in our group, we managed to all acquire new techniques and improve our skills on realtime constraints.

BIBLIOGRAPHY

[1] John R Baumgardner and Paul O Frederickson. 1985. Icosahedral discretization of the two-sphere. *SIAM Journal on Numerical Analysis* 22, 6 (1985), 1107–1115. https://doi.org/10.1137/0722066

[2] Erin Catto. 2011. Soft Constraints: Reinventing The Spring - GDC.

[3] Walaber Entertainment. 2023. Physics of JellyCar: Soft Body Physics Explained.

[4] Ricardo Marques, Christian Bouville, Mickael Ribardière, Luis Paulo Santos, and Kadi Bouatouch. 2013. Spherical Fibonacci Point Sets for Illumination Integrals. *Computer Graphics Forum* 32, 8 (2013), 134–143. https://doi.org/10.1111/cgf.12190